

SQL

ВВЕДЕНИЕ

СЕРГЕЙ РОМАНЕНКО

SQL Введение

Полное руководство по основам SQL для начинающих

Сергей Романенко

Содержание

1	Об издании	5
2	Предисловие	6
3	Учебная база данных «Игроки»	7
3.1	Структура таблицы	8
3.2	Пример записи	8
3.3	Особенности данных	8
3.4	Работа с таблицей	8
4	Типы данных	10
4.1	Целые числа	10
4.2	Строковые данные	10
4.3	Числа с дробной частью	11
4.4	Дата	12
4.5	NULL	13
4.6	Что важно запомнить	13
5	Введение в SQL	14
5.0.1	Краткая история SQL	14
6	SQL Базы данных	15
6.1	Как устроена таблица данных?	15
6.2	Что такое СУБД?	17
7	Что такое таблица в базе данных?	18
7.0.1	Разбираем на примере таблицы <code>players</code>	19
7.0.2	Как на это смотрит SQL-аналитик:	19
8	Что такое ячейка данных?	21
8.0.1	Разбираем на примере игрока ThunderStrike	21
9	Что такое запись (строка) в таблице?	23
9.0.1	Разбираем структуру строки на примере игрока AlphaKnight	23
10	Что такое столбец (поле) в таблице?	25
10.0.1	Разбираем вертикальный срез на примере столбца <code>city</code>	25
10.1	Зачем столбцам нужны типы данных?	26
11	Выборка данных (Оператор SELECT)	27
11.1	Выбираем ВСЕ столбцы из таблицы	27
11.2	Выбираем только ОПРЕДЕЛЕННЫЕ столбцы	28
12	Избавляемся от дубликатов (Ключевое слово DISTINCT)	31
12.1	Задача: Узнать список всех городов, в которых живут наши игроки	31
12.2	Полезный лайфхак: DISTINCT для нескольких столбцов	32
13	Ограничение и сдвиг вывода (LIMIT и OFFSET)	33
13.1	Вариант 1. Ограничиваем вывод с помощью LIMIT	33
13.2	Вариант 2. Постраничная навигация с помощью OFFSET	34
13.3	Вариант 3. Совмещение фильтрации WHERE и LIMIT	34
13.4	Спойлер для внимательных читателей	35
14	Сортировка данных (Оператор ORDER BY)	36
14.1	Синтаксис и главные правила	36
14.2	Сортировка по возрастанию	37
14.3	Сортировка по убыванию	37
14.4	Многоуровневая сортировка (Сортируем по нескольким столбцам)	38
15	Фильтрация данных (Оператор WHERE)	39
15.1	Примеры базовой фильтрации	39

15.2	Сложные условия и логические операторы (AND, OR, NOT)	40
15.3	Операторы фильтрации в SQL	41
15.4	Арифметические операторы в запросах	42
16	Псевдонимы (Ключевое слово AS)	44
16.1	Псевдонимы для столбцов (Переименование полей в отчете)	44
16.2	Псевдонимы для таблиц (Сокращение кода)	45
17	Группировка данных (Оператор GROUP BY)	46
17.1	Среднее значение и округление (AVG, ROUND)	46
17.2	Подсчет количества строк (COUNT)	47
17.3	Фильтрация групп (Оператор HAVING)	48
17.4	Сортировка сгруппированных данных	48
17.5	Поиск максимума и минимума (MAX, MIN)	49
17.6	Группировка по нескольким полям	50
17.7	Разница между WHERE и HAVING: золотое правило	50
18	Агрегатные функции	51
18.1	Что такое агрегатные функции и как они работают?	51
18.2	Функция COUNT (Количество)	52
18.3	Функция SUM (Сумма)	52
18.4	Функция AVG (Среднее значение)	52
18.5	Функция MIN (Минимум)	53
18.6	Функция MAX (Максимум)	53
19	Что дальше?	55
19.1	SQL Введение	55
19.2	SQL для начинающих	55
19.3	SQL Оконные функции	55
19.4	Практический тренажёр по SQL	55
19.5	Проверка знаний и подготовка к собеседованию по SQL	55
20	Об авторе	57

1 Об издании

Перед вами экземпляр книги **SQL Введение**, собранный из исходного репозитория Git.

Версия: **v1.0.4** (коммит **141e83b**, ветка **master**). Дата сборки: 2026-07-17 15:23:08 +07.

Каждая сборка создаётся напрямую из исходного кода книги, что обеспечивает воспроизводимость версии.

Исходный репозиторий: <https://github.com/Awilum/sql-introduction>

2 Предисловие

Добро пожаловать!

Вы держите в руках книгу, которая поможет сделать первые шаги в изучении языка SQL — одного из самых востребованных инструментов для работы с данными.

Сегодня SQL используют разработчики, аналитики данных, тестировщики, специалисты по бизнес-аналитике, инженеры данных и многие другие профессионалы. Независимо от того, хотите ли вы начать карьеру в IT, повысить свою квалификацию или просто разобраться, как работают базы данных, знание SQL станет важным преимуществом.

Эта книга создана как практическое руководство для начинающих. Здесь нет лишней теории и сложных академических объяснений. Каждая глава посвящена отдельной теме и сопровождается большим количеством примеров, которые помогут постепенно освоить язык запросов и понять логику работы с реляционными базами данных.

Материал выстроен по принципу «от простого к сложному». Сначала вы научитесь получать данные из таблиц, затем — фильтровать и сортировать результаты, использовать встроенные функции, выполнять вычисления и строить более сложные запросы. Каждый новый раздел опирается на знания, полученные ранее, поэтому книгу рекомендуется читать последовательно.

Чтобы обучение было максимально эффективным, старайтесь не просто читать примеры, а самостоятельно выполнять запросы, изменять их и экспериментировать. Именно практика позволяет быстрее понять принципы работы SQL и приобрести уверенность при написании собственных запросов.

Надеюсь, эта книга станет для вас хорошей отправной точкой в изучении SQL и поможет сделать первый шаг к уверенному владению одним из самых важных языков для работы с данными.

Желаю приятного чтения и успешного обучения!

3 Учебная база данных «Игроки»

В этой книге используется единая учебная база данных «Игроки».

Все примеры и SQL-запросы основаны на таблице `players`.

Если вы хотите не только читать книгу, но и выполнять все запросы самостоятельно, воспользуйтесь интерактивной SQL-песочницей, доступной в рамках курса «SQL Введение».

В песочнице вы сможете:

- выполнять любые SQL-запросы;
- изучать структуру таблицы;
- экспериментировать с данными;
- проверять собственные решения;
- закреплять материал на практике.

Введите SQL запрос

>_ SQL

1

2

3

4

5

6

7

8

9

▶ Запустить код

Отправить на проверку

Доступ к курсу и SQL-песочнице: [Записаться на курс «SQL Введение»](#)

3.1 Структура таблицы

Поле	Тип данных	Описание
id	INT	Уникальный идентификатор игрока
nickname	VARCHAR(50)	Игровой ник
email	VARCHAR(100)	Электронная почта
city	VARCHAR(50)	Город проживания
country	VARCHAR(50)	Страна проживания
level	INT	Уровень персонажа
rating	INT	Игровой рейтинг
rank_title	VARCHAR(50)	Ранг игрока (Bronze, Silver, Gold, Platinum, Diamond, Master)
guild	VARCHAR(50)	Гильдия или клан игрока
wins	INT	Количество побед
losses	INT	Количество поражений
win_rate	DECIMAL(5,2)	Процент побед
last_login	DATE	Дата последнего входа
registration_date	DATE	Дата регистрации аккаунта

3.2 Пример записи

id	nickname	city	level	rating	rank_title	guild
1	AlphaKnight	Москва	45	2100	Gold	Грифоны Эрафии

3.3 Особенности данных

При работе с таблицей обратите внимание на несколько важных моментов:

- Некоторые игроки не состоят в гильдиях. В поле `guild` для них хранится значение `NULL`.
- В таблице представлены игроки разных уровней — от новичков до опытных пользователей.
- Несколько игроков могут проживать в одном городе.
- В одной гильдии может состоять сразу несколько игроков.
- Даты регистрации и последних входов позволяют выполнять запросы по времени.
- Таблица содержит как числовые, так и текстовые данные.

3.4 Работа с таблицей

Во всех главах этой книги запросы будут выполняться к одной и той же таблице:

players

Перед началом изучения книги рекомендуется ознакомиться со структурой таблицы и понять назначение каждого столбца. Это поможет лучше понимать примеры и быстрее освоить SQL.

4 Типы данных

При создании таблицы необходимо указать, какие данные будут храниться в каждом столбце.

Для этого используются типы данных.

Тип данных определяет, какие значения можно записывать в столбец таблицы.

Рассмотрим таблицу игроков:

Столбец	Тип данных
id	INT
nickname	VARCHAR(50)
email	VARCHAR(100)
level	INT
rating	INT
win_rate	DECIMAL(5,2)
last_login	DATE
registration_date	DATE

Каждый столбец хранит определенный тип информации, поэтому для него выбирается подходящий тип данных.

4.1 Целые числа

Для хранения целых чисел используются числовые типы данных.

В нашей таблице это:

```
id INT
level INT
rating INT
wins INT
losses INT
```

Примеры значений:

level
1
25
100

Тип **INT** используется очень часто и подходит для хранения большинства целых чисел.

4.2 Строковые данные

Для хранения текста используются строковые типы данных.

В нашей таблице это:

```
nickname VARCHAR(50)
email VARCHAR(100)
city VARCHAR(50)
country VARCHAR(50)
rank_title VARCHAR(50)
guild VARCHAR(50)
```

Примеры значений:

nickname
Dragon
Shadow
Warrior77

Тип **VARCHAR** хранит строки переменной длины.

Например:

```
VARCHAR(50)
```

Число в скобках указывает максимальное количество символов, которое может храниться в поле.

Для столбца:

```
nickname VARCHAR(50)
```

можно сохранить ник длиной до 50 символов.

4.3 Числа с дробной частью

Для хранения чисел с дробной частью используются специальные типы данных.

В нашей таблице используется:

```
win_rate DECIMAL(5,2)
```

Примеры значений:

win_rate
52.30
75.50
100.00

Тип:

DECIMAL(5,2)

означает:

- всего можно хранить 5 цифр;
- из них 2 цифры находятся после десятичной точки.

Например:

99.99
12.50
100.00

Такой тип данных часто используется для процентов и денежных значений.

4.4 Дата

Для хранения даты используется тип данных:

DATE

В нашей таблице это:

last_login **DATE**
registration_date **DATE**

Примеры значений:

2025-01-15
2025-06-20
2026-01-01

Формат даты:

ГГГГ-ММ-ДД

Например:

2026-06-25

4.5 NULL

Некоторые поля могут не содержать значения.

В таком случае используется специальное значение:

NULL

Например, игрок может еще не состоять в гильдии:

nickname	guild
Dragon	NULL

Или игрок еще не получил звание:

nickname	rank_title
Shadow	NULL

NULL означает, что значение отсутствует или пока неизвестно.

Важно понимать разницу:

Значение	Смысл
NULL	значение отсутствует
“ ”	пустая строка
0	число ноль

Это разные значения.

4.6 Что важно запомнить

- Тип данных определяет, какие значения можно хранить в столбце.
- Для целых чисел в нашей таблице используется тип INT.
- Для текста используется VARCHAR.
- Для процентов побед используется DECIMAL.
- Для хранения даты используется DATE.
- NULL означает отсутствие значения.

5 Введение в SQL

SQL (Structured Query Language) — декларативный язык программирования, является стандартным языком для работы с реляционными базами данных. Он используется для создания, изменения и управления структурой базы данных, а также для выполнения запросов к базе данных и получения данных из нее.

Декларативный язык программирования — это язык программирования, в котором разработчик описывает **что нужно получить**, а не **как именно это сделать**.

SQL используется в большинстве современных **СУБД** (Системы Управления Базами Данных) такие как **MySQL**, **Microsoft SQL Server**, **Oracle** и многие другие.

5.0.1 Краткая история SQL

Язык **SQL** появился после реляционной алгебры, и его прототип был разработан в конце 70-х годов в компании **IBM Research**. Он был реализован в первом прототипе реляционной **СУБД** фирмы IBM System R. С тех пор этот язык использовался многими коммерческими **СУБД** и был настолько популярен, что постепенно стал де-факто стандартным языком манипулирования данными в реляционных **СУБД**.

Реляционная алгебра — замкнутая система операций над отношениями в реляционной модели данных.

Операции реляционной алгебры также называют реляционными операциями.

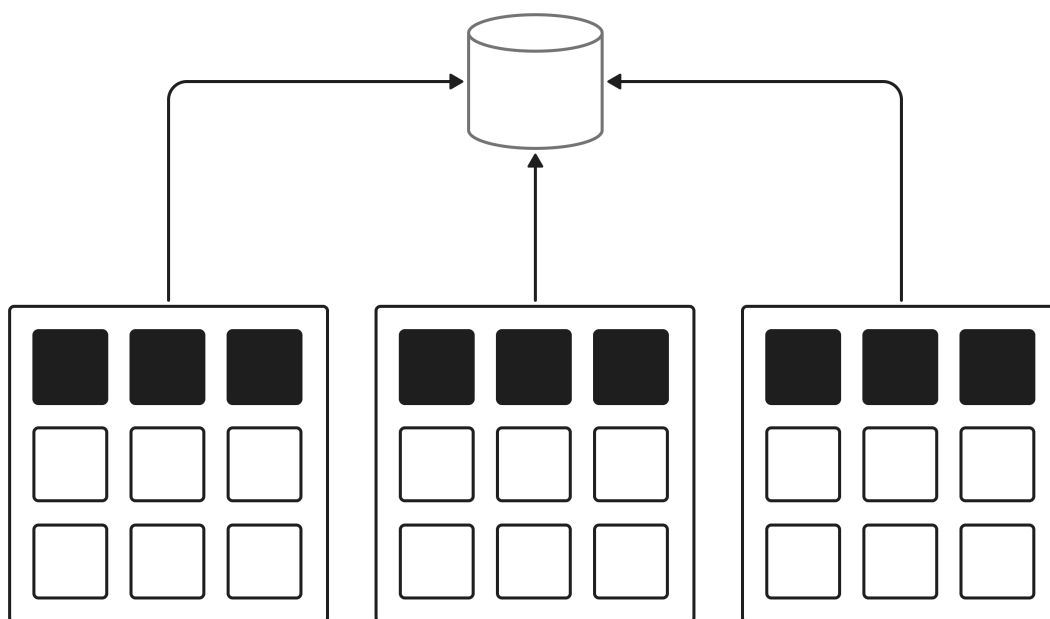
Первоначально язык назывался **SEQUEL** (Structured English Query Language), но потом слово «**English**» пропало из этого словосочетания. С одной стороны, **SQL** ориентирован на то, чтобы сделать запросы к реляционным базам данных удобными и понятными для пользователей. С другой стороны, его почти с самого начала называли «**полным языком баз данных**».

SQL включает в себя:

- средства для определения и управления схемой базы данных
- средства для определения ограничений целостности и триггеров
- средства для определения представлений БД
- средства для авторизации доступа к отношениям и их полям
- средства для определения структур физического уровня, поддерживающих эффективное выполнение запросов
- средства для определения точек сохранения транзакции и выполнения фиксации и откатов транзакций

6 SQL Базы данных

SQL Базы данных (SQL Databases) — это специальные системы для надежного хранения, организации и быстрой обработки больших объемов информации. Чтобы находить нужные сведения, изменять их или удалять, они используют единый общепринятый язык — **SQL** (*Structured Query Language* или *Язык структурированных запросов*).



В рамках книги мы будем изучать все команды и концепции на примере **учебной базы данных «Игроки»**. В ней собрана подробная статистика о киберспортсменах, их рейтингах, гильдиях и результатах матчей. Это поможет вам сразу применять теорию на понятных и практических игровых сценариях!

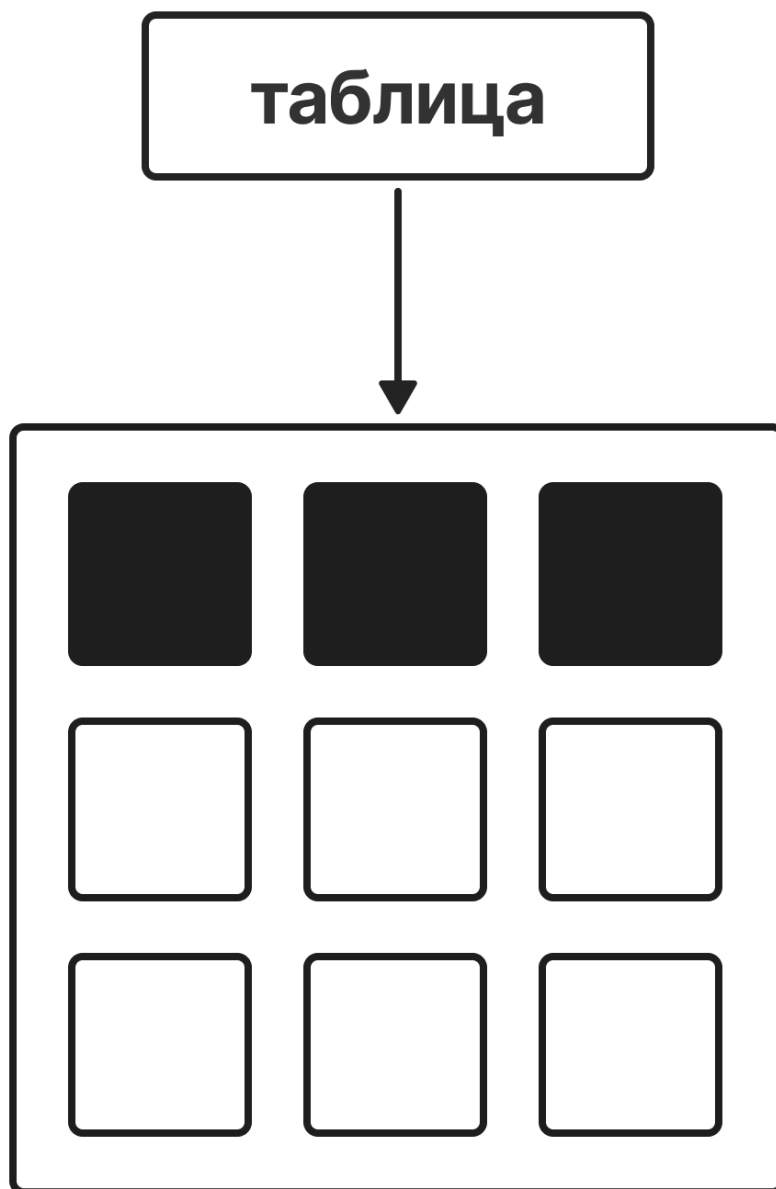
Вся информация о наших игроках и их достижениях хранится в **реляционных** базах данных. Слово «реляционный» происходит от английского *relation* (отношение, связь). Это значит, что данные внутри системы организованы в виде простых и понятных **таблиц**, которые при необходимости можно связывать друг с другом.

6.1 Как устроена таблица данных?

Структура реляционной таблицы очень похожа на привычные нам таблицы в Excel или Google Документах. Она строго состоит из строк и столбцов:

- **Столбцы (Поля)** — определяют, какие именно характеристики объекта мы записываем. Каждый столбец имеет свое имя и определенный тип данных (текст, целое число, дробное число и т.д.). В нашей таблице `players` это такие поля, как `nickname`, `level`, `rating` или `registration_date`.

- **Строки (Записи)** — это конкретные примеры объектов. В нашем случае каждая строка — это анкета отдельного киберспортсмена со всеми его игровыми показателями.



Использование баз данных вместо обычных текстовых блокнотов или разрозненных файлов дает разработчику (и аналитику!) колоссальные преимущества:

- **Универсальный язык запросов.**

С помощью SQL вы можете написать один простой запрос и например мгновенно отфильтровать 50 игроков по нужному вам уровню, проценту побед или принадлежности к определенной гильдии.

- **Целостность данных и отсутствие дублирования.**

Базы данных позволяют связывать таблицы между собой. Например, можно вынести детальное описание игровых кланов в отдельную таблицу, избегая повторения одной и той же информации о гильдии для каждого участника.

- **Быстродействие.**

Благодаря созданию специальных «указателей» (индексов), поиск нужного игрока среди множества записей в базе данных занимает доли секунды.

- **Безопасность и контроль.**

База данных четко контролирует, кто имеет право просматривать конфиденциальную информацию, а кто — изменять.

6.2 Что такое СУБД?

Сама по себе база данных — это просто упорядоченное хранилище файлов на диске.

Чтобы мы могли работать с ней с помощью языка SQL, нам нужна специальная программа.

СУБД (Система Управления Базами Данных) или на английском **RDBMS** (*Relational Database Management System*) — это программное обеспечение, которое позволяет создавать базы данных, наполнять их, защищать и выполнять к ним SQL-запросы.

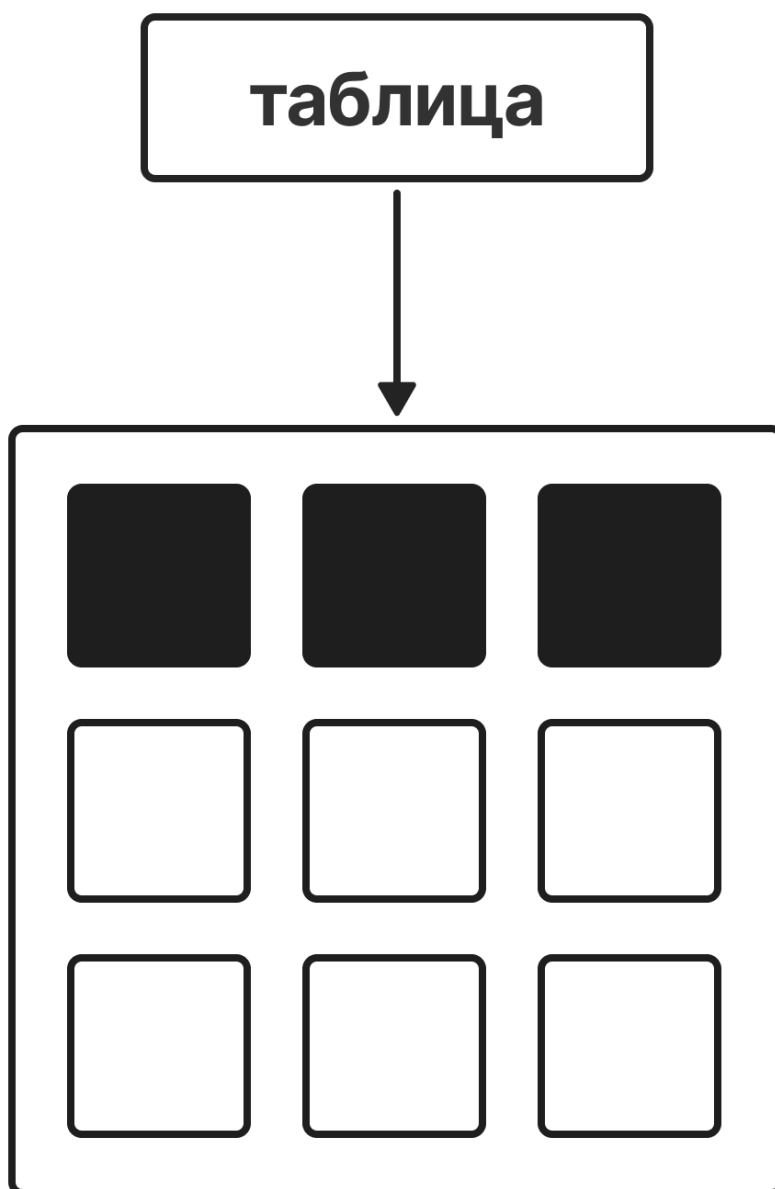
Существует множество популярных СУБД. Они могут немного отличаться интерфейсом и дополнительными возможностями, но все они используют и понимают стандартный язык SQL. К самым известным относятся:

- **MySQL** (именно её мы будем использовать в книге!)
- PostgreSQL
- SQLite
- Oracle
- Microsoft SQL Server

7 Что такое таблица в базе данных?

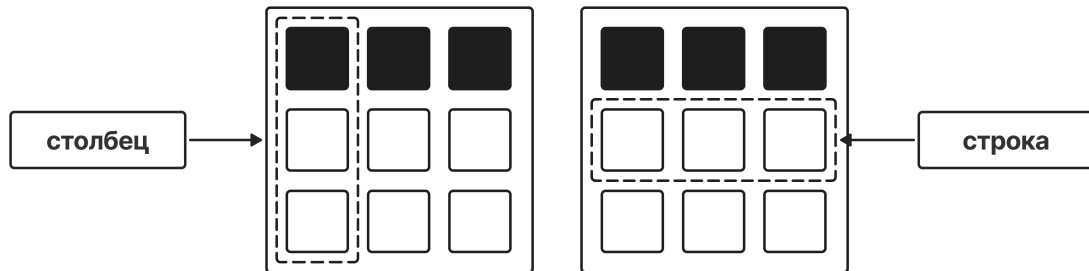
Таблица (table) в SQL — это основной контейнер для хранения информации в реляционной базе данных.

Представьте её как продвинутую и очень строгую таблицу в Excel или Google Таблицах.



Каждая таблица состоит из **столбцов** и **строк**:

- **Столбец (поле / column)** — определяет, какой именно тип информации хранится. Например, никнейм, уровень персонажа или дата его регистрации в игре. У каждого столбца есть свое фиксированное имя и строго определенный тип данных.
- **Строка (запись / row)** — это один конкретный объект в базе данных. Сколько строк в таблице — столько реальных геймеров со всей их статистикой мы в неё записали.



7.0.1 Разбираем на примере таблицы `players`

В нашей книге мы работаем с таблицей `players`. Она создана для того, чтобы собирать, обновлять и анализировать подробную статистику участников игрового сообщества, их успехи, гильдии и турнирные ранги.

Давайте посмотрим, как устроена часть этой таблицы изнутри (первые три записи):

id	nickname	city	level	rating	rank_title	guild
1	AlphaKnight	Москва	45	2100	Gold	Грифоны Эрафии
2	CatQueen	Санкт-Петербург	12	850	Bronze	NULL
3	ThunderStrike	Новосибирск	89	3450	Diamond	Черные Драконы Нигона

7.0.2 Как на это смотрит SQL-аналитик:

1. Каждый столбец задает характеристику игрока.

Столбец `nickname` всегда хранит текст (игровое имя), а `rating` или `level` — строго целое число (показатели прогресса).

2. Каждая строка — это один конкретный игрок со всеми его условиями.

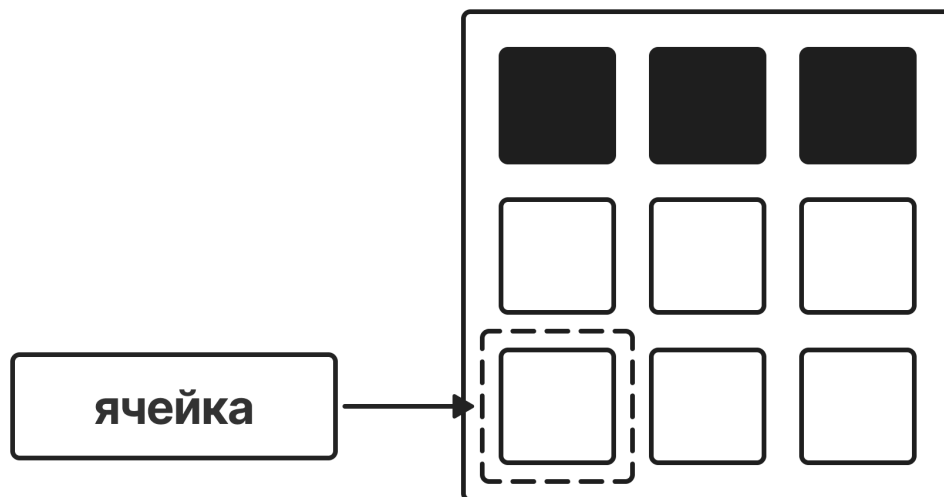
Например, строка №1 целиком описывает геймера с ником *AlphaKnight*: он живет в Москве, достиг 45-го уровня, имеет 2100 очков рейтинга, носит звание «Gold» и состоит в клане «Грифоны Эрафии». А у игрока *CatQueen* под номером 2 в поле `guild` указано значение *NULL* — это значит, что на данный момент она не состоит ни в одной игровой гильдии.

Важное правило:

В реляционных таблицах изначальный порядок строк не имеет принципиального значения. База данных может выдать их при запросе в любом случайном порядке, пока мы сами жестко не попросим её отсортировать данные. Но об этом — в следующих уроках!

8 Что такое ячейка данных?

Ячейка (item / cell) в базе данных — это самый маленький, неделимый элемент данных, который может быть сохранен в таблице. Она находится на **пересечении одной конкретной строки и одного конкретного столбца**.



Каждая ячейка содержит только **одно значение** и строго подчиняется правилам своего столбца:

- Если столбец предназначен для текста (например, игровые никнеймы), в его ячейку нельзя записать дату.
- Если столбец хранит целые числа (например, количество побед), там не может оказаться текст или дробный процент.

8.0.1 Разбираем на примере игрока ThunderStrike

Давайте посмотрим на одну конкретную запись из нашей игровой базы данных `players`, которая описывает опытного киберспортсмена под ником *ThunderStrike*:

id	nickname	city	level	rating	rank_title	guild	wins	win_rate
3	ThunderStrike	Новосибирск	89	3450	Diamond	Черные Драконы Нигона	540	56.84

В этой строке каждая отдельная ячейка несет свой смысл и имеет свой строго определенный тип данных:

- В ячейке столбца `nickname` хранится **строка текста** — `'ThunderStrike'`.
- В ячейке столбца `city` хранится **строка текста** — `'Новосибирск'`.

- В ячейке столбца `level` хранится **целое число** — `89` (текущий игровой уровень).
- В ячейке столбца `rating` хранится **целое число** — `3450` (очки соревновательного рейтинга).
- В ячейке столбца `win_rate` хранится **дробное число** — `56.84` (процент успешных матчей).

Обратите внимание:

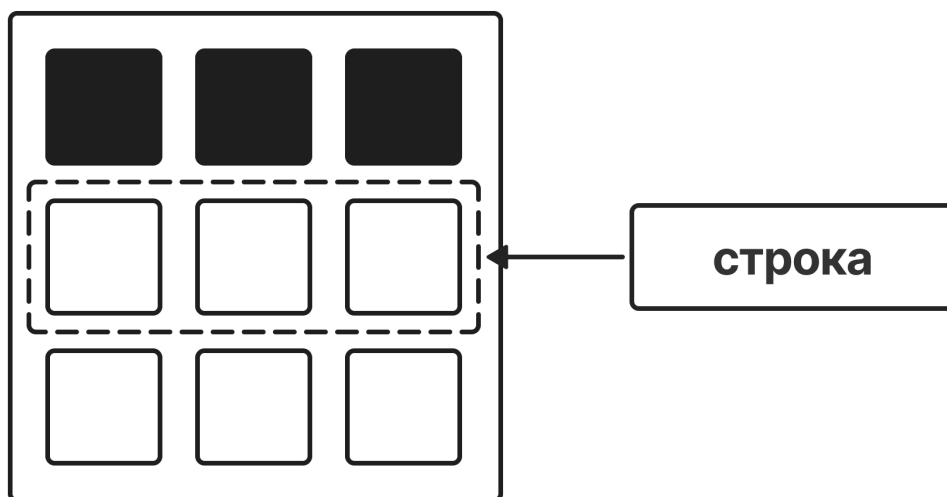
В SQL-базах данных пустая ячейка — это тоже значение. Если игрок только зарегистрировался, еще не успел вступить ни в один игровой клан или не получил турнирное звание, в такую ячейку временно записывается специальное значение **NULL** (обозначающее отсутствие данных).

Например, у игрока *CatQueen* (id 2) в ячейке столбца `guild` стоит именно *NULL*.

id	nickname	city	level	rating	rank_title	guild
2	CatQueen	Санкт-Петербург	12	850	Bronze	NULL

9 Что такое запись (строка) в таблице?

Запись (record) или **строка (row)** в SQL-базе данных — это горизонтальный срез таблицы, который содержит все данные для одной конкретной единицы информации. В реляционных базах данных каждая строка описывает один самостоятельный и целостный объект.



В нашей таблице `players` каждая отдельная строка представляет собой **одного конкретного игрока** со своим набором уникальных характеристик (столбцов): `id`, `nickname`, `email`, `city`, `level`, `rating` и так далее. Сколько геймеров зарегистрировано в нашей игре — столько строк и будет в таблице.

9.0.1 Разбираем структуру строки на примере игрока AlphaKnight

Давайте выделим из нашей базы данных самую первую строку и посмотрим, из каких полей она состоит.

В этой строке собраны данные, которые относятся **строго к одному пользователю** и описывают его текущий игровой профиль:

- **id:** 1 (уникальный номер этой записи в системе).
- **nickname:** AlphaKnight (уникальное имя в игре).
- **email:** alpha_knight@yandex.ru (контактная почта).
- **city:** Москва (город проживания).
- **country:** Россия (страна игрока, установленная по умолчанию).
- **level:** 45 (текущий уровень персонажа).
- **rating:** 2100 (очки соревновательного рейтинга).

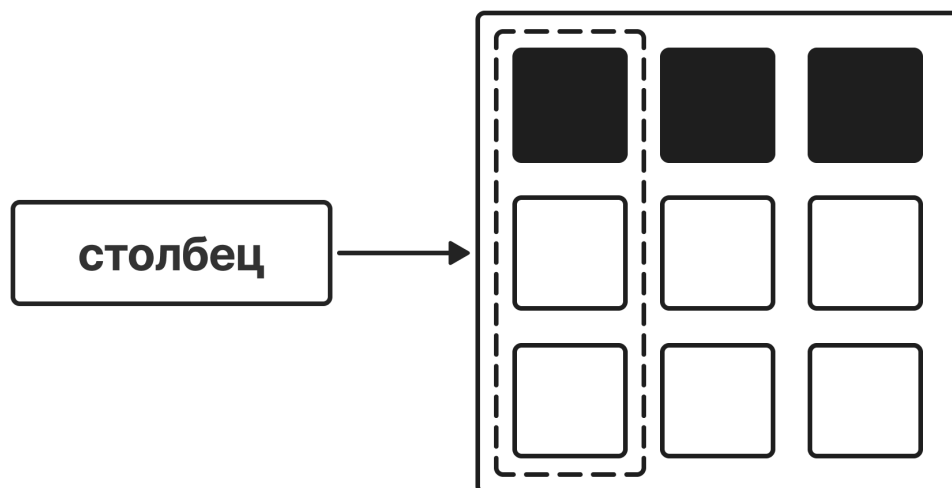
- **rank_title:** Gold (текущее турнирное звание).
- **guild:** Грифоны Эрафии (игровой клан, в котором состоит пользователь).
- **wins:** 120 (количество побед в матчах).
- **losses:** 80 (количество поражений).
- **win_rate:** 60.00 (процент успешных игр).
- **last_login:** 2026-06-20 (дата последнего посещения игры).
- **registration_date:** 2025-01-15 (день, когда игрок создал аккаунт).

Ключевое правило баз данных:

Все данные внутри одной строки жестко связаны между собой. Мы не можем случайно перемешать количество побед игрока *AlphaKnight* с никнеймом или почтой игрока *CatQueen* — СУБД строго следит за целостностью и неделимостью каждой строки.

10 Что такое столбец (поле) в таблице?

Столбец (column) в SQL базе данных — это вертикальный срез таблицы, который используется для хранения данных одного строго определенного типа.



Каждый столбец в таблице обязательно имеет:

1. **Свое уникальное имя** (например, `nickname` или `rating`).
2. **Тип данных** — ограничение, которое указывает базе, что именно здесь можно хранить (текст, целое число, дробь или дату).

Лайфхак для новичков:

Термины «**столбец**» и «**поле**» в SQL используются как синонимы. Это абсолютно одно и то же.

10.0.1 Разбираем вертикальный срез на примере столбца `city`

Давай посмотрим, как выглядит отдельный столбец `city` из нашей таблицы `players`.

Он имеет тип данных `VARCHAR` (строка текста) и хранит исключительно названия городов проживания наших игроков:

city
Москва
Санкт-Петербург
Новосибирск
Екатеринбург
Омск
Казань
Нижний Новгород

10.1 Зачем столбцам нужны типы данных?

Тип данных защищает таблицу от хаоса и ошибок.

Давай посмотрим на три разных столбца из нашей базы `players` :

nickname	rating	registration_date
----------	--------	-------------------

- **nickname** (Тип: Текст / `VARCHAR`) — хранит слова.

База данных понимает, что эти значения можно переводить в верхний регистр, проверять на длину или искать в них определенные буквы.

- **rating** (Тип: Целое число / `INT`) — хранит только числа без запятой.

База знает, что их можно складывать, находить среднее соревновательное значение или искать игроков, у которых рейтинг выше 2000. Сюда физически нельзя записать слово «Профи».

- **registration_date** (Тип: Дата / `DATE`) — хранит строго даты в формате `ГГГГ-ММ-ДД`.

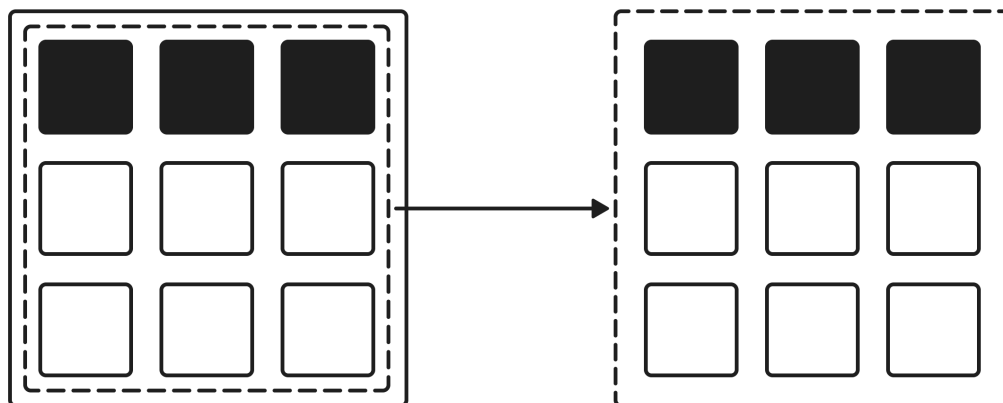
База понимает, какой пользователь зарегистрировался в игре раньше, а какой позже.

Именно благодаря столбцам мы сможем говорить с базой данных на языке аналитики: например, попросить её «*посчитать среднее значение по столбцу Процент побед (`win_rate`)*» или «*оставить только те строки, где в столбце Звание (`rank_title`) написано „Gold“*».

11 Выборка данных (Оператор SELECT)

Чтобы заставить базу данных показать нам информацию, мы пишем ей специальную команду — **запрос (query)**.

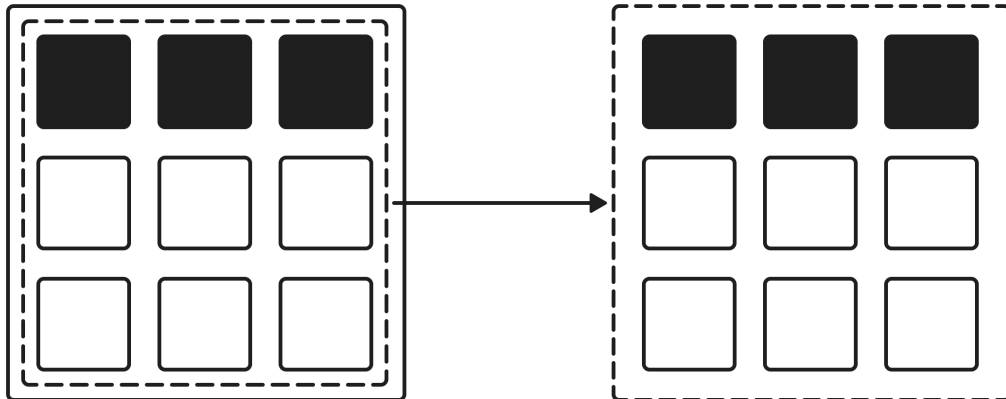
Главный инструмент для этого в SQL — ключевое слово **SELECT** (выбрать).



С помощью **SELECT** мы можем забрать из таблицы либо вообще все данные, либо только нужные нам столбцы, либо отфильтровать строки по какому-то условию.

11.1 Выбираем ВСЕ столбцы из таблицы

Представь, что тебе нужно просто посмотреть на всю нашу таблицу целиком, чтобы изучить зарегистрированных игроков. Чтобы не перечислять вручную все 14 названий столбцов, в SQL используют знак «звёздочка» (*****). Она означает «покажи мне все поля, которые есть в таблице».



Наш запрос:

```
SELECT * FROM players;
```

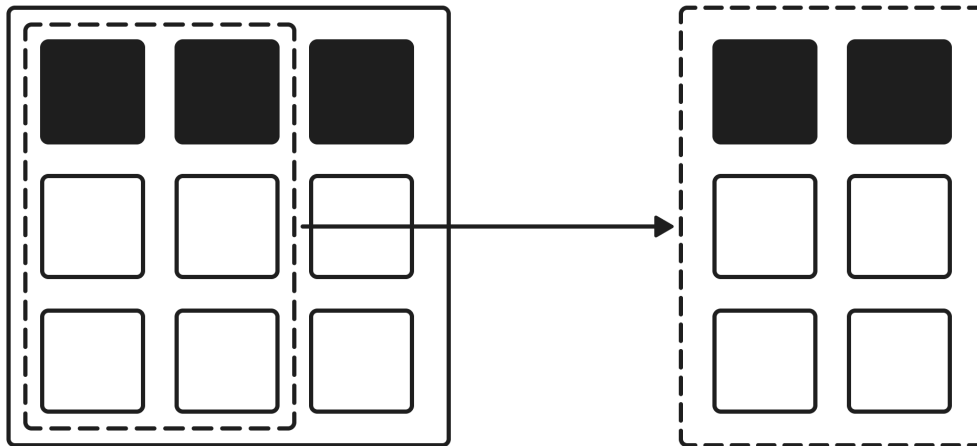
Результат:

База данных выведет полную таблицу со всеми 50 строками и всеми столбцами (`id`, `nickname`, `email`, `city`, ...).

11.2 Выбираем только ОПРЕДЕЛЕННЫЕ столбцы

Чаще всего аналитику или разработчику не нужен весь массив данных.

Например, твоя задача — составить простой дашборд со списком никнеймов игроков, их текущим уровнем и рейтингом, а почта, города и даты входа сейчас не важны.



Чтобы убрать лишний «шум», мы перечисляем нужные столбцы через запятую сразу после слова `SELECT` :

Наш запрос:

```
SELECT nickname,  
        level,  
        rating  
FROM players;
```

SQL вернет аккуратную «вырезку» из таблицы.

В ней по-прежнему будут все 50 игроков, но отображаемая информация станет компактной.

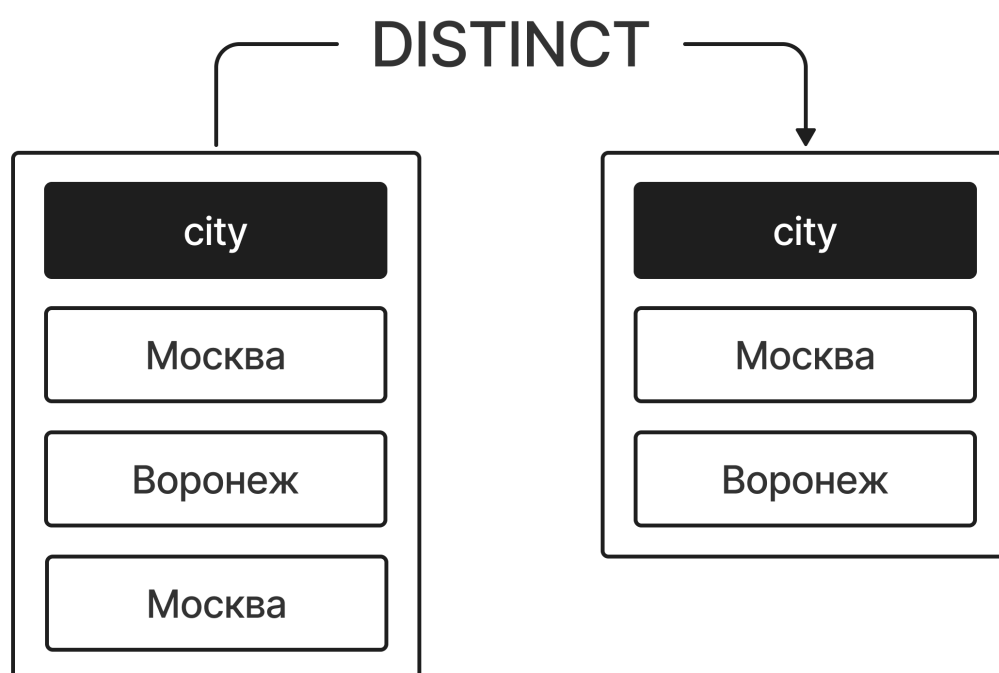
Результат (срез, первые 10 строк):

nickname	level	rating
AlphaKnight	45	2100
CatQueen	12	850
ThunderStrike	89	3450
AliceFox	31	1500
MaxDrive	55	2400
NightWolf	73	2900
StarDust	22	1150
SteelKnight	61	2650
CyberKing	40	1950
PhoenixRF	50	2200

12 Избавляемся от дубликатов (Ключевое слово DISTINCT)

Когда ты работаешь с большими базами данных, значения в столбцах неизбежно повторяются. Например, в нашей таблице `players` находится 50 геймеров, но крупных городов или игровых рангов в системе гораздо меньше.

Если мы сделаем обычный запрос `SELECT city FROM players;`, база данных честно выведет нам 50 строчек, где слова «Москва», «Санкт-Петербург» или «Казань» повторятся многократно. Читать такой отчет крайне неудобно.



Чтобы убрать этот «информационный шум» и оставить только уникальные значения, в SQL используется ключевое слово `DISTINCT` (отдельный / уникальный).

12.1 Задача: Узнать список всех городов, в которых живут наши игроки

Мы хотим получить чистый список городов без повторений, чтобы понимать географию нашей игровой базы. Для этого мы ставим слово `DISTINCT` строго **после** `SELECT`, но **перед** именем столбца:

Наш запрос:

```
SELECT DISTINCT city
FROM players;
```

Как это работает:

Упрощённо можно представить, что база данных просматривает все значения столбца и оставляет только уникальные. Поэтому в результате каждый город появляется только один раз.

Результат (уникальные города из 50 строк таблицы, **срез, первые 10 строк**):

city
Москва
Санкт-Петербург
Новосибирск
Екатеринбург
Омск
Казань
Нижний Новгород
Челябинск
Самара
Ростов-на-Дону

12.2 Полезный лайфхак: DISTINCT для нескольких столбцов

Если указать несколько столбцов через запятую после `DISTINCT`, например:

```
SELECT DISTINCT rank_title, guild
FROM players;
```

SQL будет искать уникальные комбинации значений в этих столбцах.

Мы увидим, какие сочетания званий и кланов встречаются в базе данных, но каждая комбинация будет показана только один раз.

Например:

- Gold — Грифоны Эрафии
- Platinum — Черные Драконы Нигона
- Gold — Золотые Единороги Авли

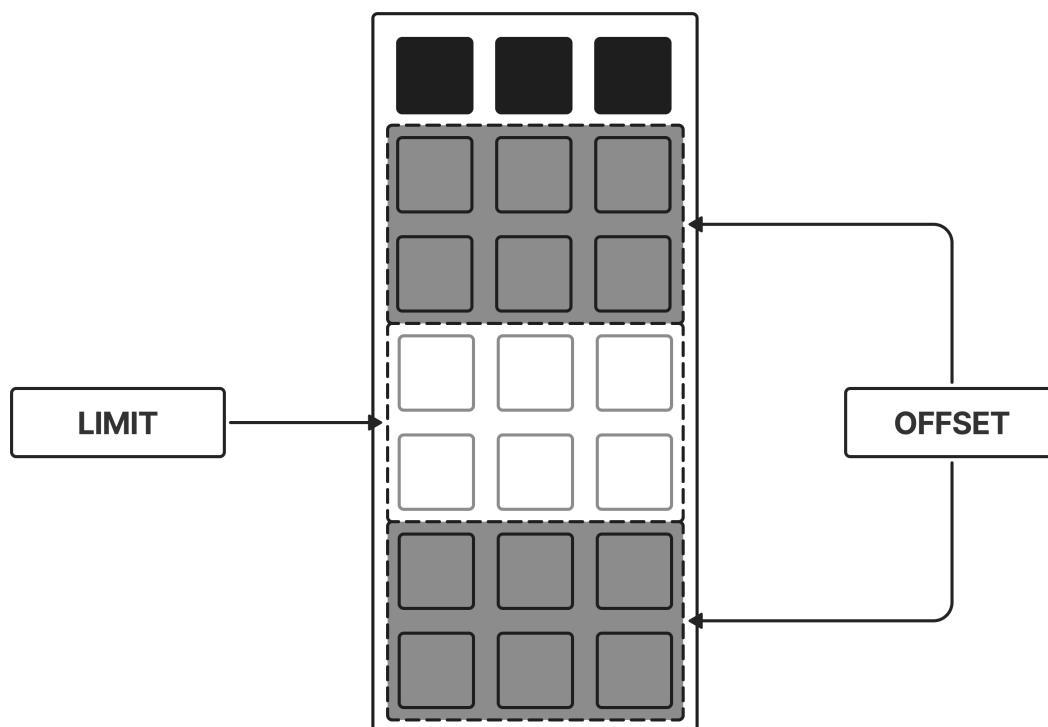
Если одна и та же пара значений встречается в таблице несколько раз, в результат она попадёт только один раз.

Запомни правило:

ключевое слово `DISTINCT` всегда пишется сразу после `SELECT`. Оно проверяет уникальность всей комбинации выбранных столбцов, а не каждого столбца по отдельности.

13 Ограничение и сдвиг вывода (LIMIT и OFFSET)

Представь, что в интерфейсе нашей игры открывается вкладка «Рейтинг лидеров». В базе данных `players` хранится большое количество пользователей. Если вывалить на экран геймера сразу все 50 (или тысячи!) строк, интерфейс будет долго грузиться, а на экране начнется каша.



Чтобы управлять объёмом выдачи, в SQL используются два мощных инструмента:

- **LIMIT** — указывает *максимальное количество строк*, которое нужно вернуть.
- **OFFSET** — указывает, *сколько строк нужно пропустить* с самого начала, прежде чем выводить результат.

13.1 Вариант 1. Ограничиваем вывод с помощью LIMIT

Допустим, тебе нужно сделать небольшой виджет для главного меню игры: «ТОП-3 игрока по соревновательному рейтингу». Нам не нужна вся база, нам нужны только первые три строки после сортировки.

Наш запрос:

```
SELECT nickname, level, rating
FROM players
ORDER BY rating DESC
LIMIT 3;
```

Результат:

База данных сначала выстроит игроков по рейтингу от самого высокого к самому низкому, а затем оставит только три верхние позиции этого списка.

nickname	level	rating
AdmiralB	91	3600
ThunderStrike	89	3450
SiberianLion	82	3200

13.2 Вариант 2. Постраничная навигация с помощью OFFSET

А теперь классическая задача из веб-разработки и геймдева: нужно сделать постраничный вывод рейтинга (пагинацию). На первой странице таблицы лидеров мы показали первые 3 игрока. А как показать вторую страницу (места с 4 по 6)? Нам нужно *пропустить* первые 3 строки и забрать *следующие* 3.

Для этого мы добавляем **OFFSET 3** :

Наш запрос:

```
SELECT nickname, level, rating
FROM players
ORDER BY rating DESC
LIMIT 3 OFFSET 3;
```

Результат:

SQL взял тот же список, перешагнул через лидеров (*ThunderStrike*, *AlexMega* и *GhostRider*) и выдал следующие по порядку три строки:

nickname	level	rating
MountainQueen	77	3100
AltaiShaman	72	2980
ForestHunter	70	2950

13.3 Вариант 3. Совмещение фильтрации WHERE и LIMIT

Ты можешь легко комбинировать ограничение строк с фильтрацией.

Например, задача: «Покажи мне 2 любых опытных игроков из города Москва, у которых уровень персонажа уже больше 40».

Наш запрос:

```
SELECT nickname, city, level
FROM players
WHERE city = 'Москва' AND level > 40
LIMIT 2;
```

Результат:

База данных сначала отфильтрует таблицу по условию `WHERE`, оставив только москвичей старше 40 уровня, а из получившегося списка вернет только первые две строки:

nickname	city	level
AlphaKnight	Москва	45
MoscowGuy	Москва	42

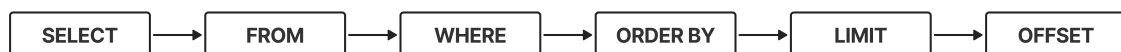
13.4 Спойлер для внимательных читателей

В примерах выше мы использовали команды `WHERE` (для фильтрации игроков по условиям) и `ORDER BY` (для сортировки по рейтингу). Не переживай, если они пока кажутся тебе сложными — в этом уроке они нужны только для того, чтобы показать красоту и пользу работы `LIMIT`.

Настоящую магию сортировки (`ORDER BY`) и точечной фильтрации данных (`WHERE`) мы во всех деталях разберем в следующих уроках. Всему свое время!

Строгий порядок команд в MySQL:

Запомни, как правильно выстраивать цепочку операторов, иначе база данных выдаст синтаксическую ошибку:



14 Сортировка данных (Оператор ORDER BY)

Когда мы просто выбираем данные из таблицы, MySQL может возвращать строки в произвольном порядке.

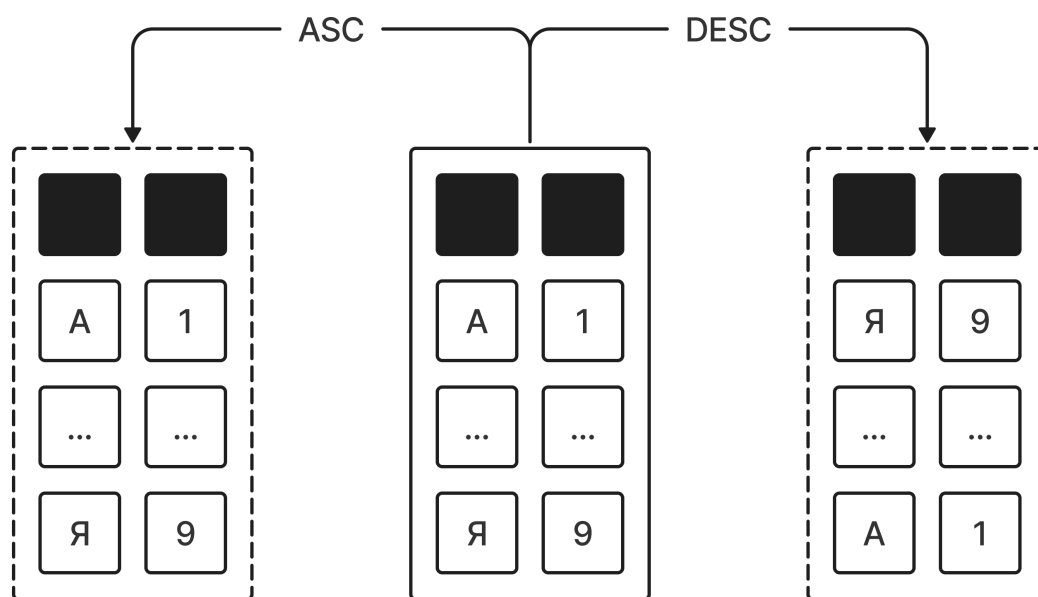
Иногда это может совпадать с порядком добавления записей, но это не гарантируется.

Но разработчику, который анализирует статистику, такой порядок обычно не подходит. Например, ему нужно отсортировать игроков: например, по уровню, рейтингу или проценту побед.

За управление порядком строк в SQL отвечает оператор `ORDER BY` — оператор сортировки результатов запроса.

14.1 Синтаксис и главные правила

Мы указываем `ORDER BY` в самом конце запроса (но перед `LIMIT` и `OFFSET`, если они есть) и пишем имя столбца, по которому хотим навести порядок.



У сортировки есть два направления:

- **ASC** (от *ascending*) — по возрастанию (от меньшего к большему, от А до Я).
Используется по умолчанию, если вообще ничего не писать после имени поля.
- **DESC** (от *descending*) — по убыванию (от большего к меньшему, от Я до А).

14.2 Сортировка по возрастанию

Давай поможем разработчику найти игроков с самым низким уровнем, чтобы отправить им предложение пройти обучающий квест. Отсортируем их по столбцу `level` от меньшего к большему.

Наш запрос:

```
SELECT nickname, city, level
FROM players
ORDER BY level ASC;
```

Так как `ASC` — это режим по умолчанию, мы могли бы просто написать `ORDER BY level`. Результат был бы точно таким же.

Результат (срез, первые 5 строк):

База данных выставит на первые строчки пользователей с минимальным игровым прогрессом:

nickname	city	level
PrinceX	Владимир	5
KazanCat	Казань	8
CatQueen	Санкт-Петербург	12
LipetskBoy	Липецк	14
NorthLight	Тула	15

14.3 Сортировка по убыванию

Например, нам нужно составить актуальный список лидеров для турнирной таблицы. Уровень персонажа сейчас не важен, критически важны очки соревновательного рейтинга (`rating`). Сортируем по убыванию (`DESC`).

Наш запрос:

```
SELECT nickname, rank_title, rating
FROM players
ORDER BY rating DESC;
```

Результат (срез, первые 5 строк):

Наверху окажутся киберспортсмены с высокими показателями рейтинга:

nickname	rank_title	rating
AdmiralB	Master	3600
ThunderStrike	Diamond	3450
SiberianLion	Diamond	3200
MountainQueen	Diamond	3100
AltaiShaman	Platinum	2980

14.4 Многоуровневая сортировка (Сортируем по нескольким столбцам)

А что если мы хотим сгруппировать игроков по городам (в алфавитном порядке), а уже **внутри каждого города** выявить лидеров по соревновательному рейтингу (от большего к меньшему)?

В SQL можно указать несколько столбцов через запятую. База данных начнет сортировать по первому, а если значения совпадут — применит правило для второго.

Наш запрос:

```
SELECT city, nickname, rating
FROM players
ORDER BY city ASC,
         rating DESC;
```

Результат (пример среза по городам, первые 5 строк):

SQL сначала отсортирует города по алфавиту. Но так как игроков из одного города в базе может быть много, внутри блока каждого города пользователи выстроятся по рейтингу вниз:

city	nickname	rating
Астрахань	Fisherman	1300
Барнаул	AltaiShaman	2980
Брянск	RangerX	2350
Владивосток	AmurTiger	2500
Владимир	PrinceX	500

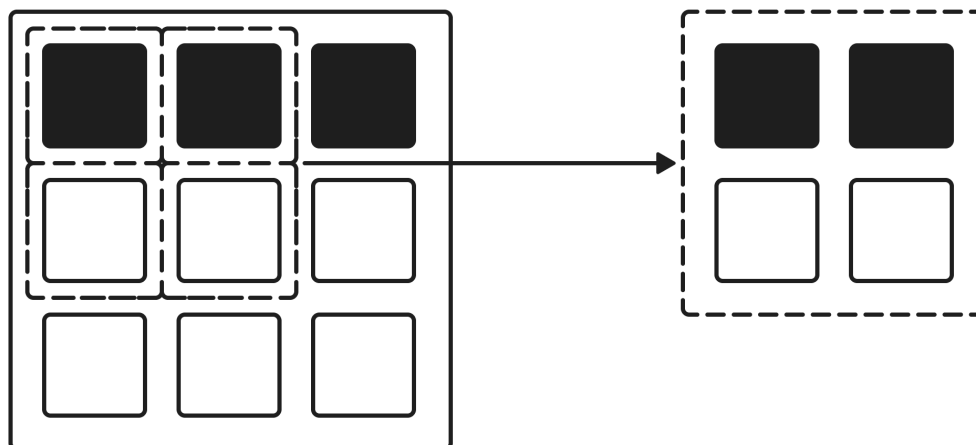
Как это читать:

База данных сначала рассматривает строки: «у меня несколько игроков из Воронежа — город одинаковый, значит смотрю на второй критерий: `rating DESC`. Поэтому более сильные игроки оказываются выше».

Затем начинается следующий город, и правило сортировки по рейтингу снова применяется внутри новой группы.

15 Фильтрация данных (Оператор WHERE)

Фильтрация данных в SQL позволяет выбрать только те строки из таблицы, которые удовлетворяют определенным условиям. Для отсеечения лишних записей используется оператор **WHERE**.



В нашей таблице `players` хранится 50 игроков со всей России. Давайте разберем, как с помощью фильтрации находить среди них нужных пользователей.

15.1 Примеры базовой фильтрации

Например, если мы хотим выбрать игроков, которые зарегистрировались в игре после 1 января 2025 года, мы пишем следующий запрос:

```
SELECT nickname, level, registration_date
FROM players
WHERE registration_date > '2025-01-15';
```

Результат (срез, первые 5 строк):

nickname	level	registration_date
CatQueen	12	2026-05-10
AliceFox	31	2025-11-03
MaxDrive	55	2025-03-14
StarDust	22	2026-02-18
SteelKnight	61	2025-05-20

Если мы хотим выбрать только тех опытных игроков, у которых уровень строго выше 40, а в поле звания (`rank_title`) указано „Gold“, запрос будет выглядеть так:

```
SELECT nickname, level, rank_title
FROM players
WHERE level > 40
AND rank_title = 'Gold';
```

Результат:

nickname	level	rank_title
AlphaKnight	45	Gold
MaxDrive	55	Gold
PhoenixRF	50	Gold
AmurTiger	58	Gold
SeaWolf	48	Gold
StormBringer	52	Gold
MoscowGuy	42	Gold
CossackRider	57	Gold
YarBear	44	Gold
FalconScout	49	Gold
ShieldWall	41	Gold
RangerX	53	Gold
MagnetPoint	46	Gold
Lia	52	Gold

15.2 Сложные условия и логические операторы (AND, OR, NOT)

С помощью оператора `WHERE` можно составлять многоуровневые запросы, комбинируя условия через логические связи:

- **AND** — должны выполняться **оба** условия одновременно.
- **OR** — должно выполняться **хотя бы одно** из условий.
- **NOT** — инвертирует (отрицает) условие.

Например, задача: «Покажи мне игроков из гильдий „Грифоны Эрафии“ или „Фениксы Конфлюкса“, у которых соревновательный рейтинг превышает 2000 очков и которые заходили в игру после 1 июня 2026 года».

```
SELECT nickname, guild, rating, last_login
FROM players
WHERE (guild = 'Грифоны Эрафии' OR guild = 'Фениксы Конфлюкса')
AND rating > 2000
AND last_login > '2026-06-01';
```

В данном запросе мы использовали круглые скобки `()` для обозначения приоритета. Сначала база данных проверит гильдию (подходит ли под одно из двух названий), а затем применит строгие ограничения по рейтингу и дате последнего визита.

Результат:

nickname	guild	rating	last_login
AlphaKnight	Грифоны Эрафии	2100	2026-06-20
MaxDrive	Грифоны Эрафии	2400	2026-06-21
BalticStorm	Грифоны Эрафии	2800	2026-06-20
SeaWolf	Грифоны Эрафии	2050	2026-06-20
MountainQueen	Фениксы Конфлюкса	3100	2026-06-21
Lia	Фениксы Конфлюкса	2380	2026-06-21

Если использовать `NOT`, мы можем легко исключить ненужное. Например, найдем игроков, которые зарегистрировались **не** в январе и **не** в феврале 2025 года:

```
SELECT nickname, registration_date
FROM players
WHERE registration_date NOT IN ('2025-01-15', '2025-02-10');
```

Здесь оператор `IN` проверяет совпадение со списком значений, а `NOT IN` — отсекает их.

15.3 Операторы фильтрации в SQL

В операторе `WHERE` вы можете использовать богатый набор инструментов для точечного поиска:

- **Равенство и неравенство (`=`, `<>`)**

```
SELECT * FROM players WHERE rank_title = 'Silver';
SELECT * FROM players WHERE city <> 'Москва'; -- все города, кроме Москвы
```

- **Сравнение (`>`, `<`, `>=`, `<=`)**

```
SELECT * FROM players WHERE level >= 50;  
SELECT * FROM players WHERE wins < 10;
```

- Проверка диапазона (**BETWEEN** .. **AND**)

```
SELECT * FROM players WHERE rating BETWEEN 1500 AND 2500; -- включает границы
```

- Поиск по шаблону текста (**LIKE** , **NOT LIKE**)

Знак `%` означает любое количество любых символов. Например, найдем все яндексовские почты:

```
SELECT * FROM players WHERE email LIKE '%@yandex.ru';
```

- Проверка на пустые значения (**IS NULL** , **IS NOT NULL**)

```
SELECT * FROM players WHERE guild IS NULL; -- игроки без клана
```

15.4 Арифметические операторы в запросах

В SQL можно выполнять математические вычисления прямо «на лету» во время выборки данных. Для этого используются стандартные знаки: `+` (сложение), `-` (вычитание), `*` (умножение), `/` (деление) и `%` (деление по модулю/остаток).

Давай посмотрим, как это применимо на практике к нашей таблице:

- **Сложение (+)** — посчитаем общее число сыгранных матчей (победы + поражения):

```
SELECT nickname, wins, losses,  
       (wins + losses) AS total_games  
FROM players  
WHERE level > 30;
```

- **Вычитание (-)** — узнаем, на сколько побед у игрока больше, чем поражений:

```
SELECT nickname,  
       (wins - losses) AS win_difference  
FROM players  
WHERE wins > losses;
```

- **Умножение** (`*`) — представим, что за каждую победу дают по 10 бонусных монет. Посчитаем призовой баланс:

```
SELECT nickname, wins,  
       (wins * 10) AS bonus_gold  
FROM players  
WHERE wins >= 100;
```

- **Деление** (`/`) — узнаем среднее количество побед на один уровень персонажа:

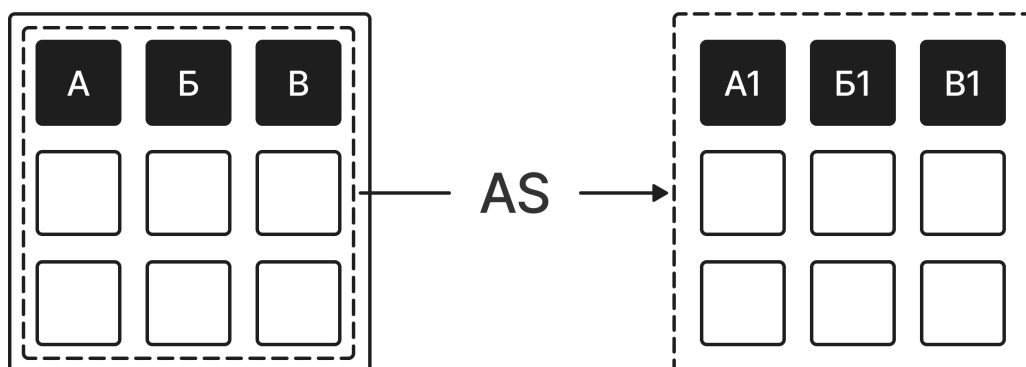
```
SELECT nickname, level, wins,  
       (wins / level) AS wins_per_level  
FROM players  
WHERE level > 0;
```

Обрати внимание:

В вычисляемых полях мы использовали ключевое слово `AS` (например, `AS total_games`). Это **псевдонимы** (`aliases`). Они дают красивое и понятное название новому виртуальному столбцу в итоговом отчете, при этом сама исходная таблица в базе данных никак не изменяется!

16 Псевдонимы (Ключевое слово AS)

Alias (псевдоним) — это временное имя, которое присваивается столбцу или таблице в SQL-запросе. Псевдонимы делают итоговый отчет более понятным для человека, а сам код запроса — более компактным и удобным для чтения, особенно если оригинальные названия полей или таблиц слишком длинные.



В SQL псевдонимы создаются с помощью ключевого слова **AS**.

16.1 Псевдонимы для столбцов (Переименование полей в отчете)

Когда мы выводим данные для аналитиков или в интерфейс приложения, технические названия столбцов вроде `nickname` или `rank_title` могут выглядеть не совсем презентабельно. Мы можем переименовать их «на лету» прямо в блоке `SELECT`.

Наш запрос:

```
SELECT id AS player_id,  
       nickname AS player_name,  
       level AS player_level,  
       rank_title AS player_rank,  
       rating AS competitive_rating  
FROM players;
```

Результат (срез, первые 5 строк):

База данных вернет привычные данные из таблицы `players`, но заголовки столбцов в итоговой таблице изменятся на те, что мы указали после `AS`:

player_id	player_name	player_level	player_rank	competitive_rating
1	AlphaKnight	45	Gold	2100
2	CatQueen	12	Bronze	850
3	ThunderStrike	89	Diamond	3450
4	AliceFox	31	Silver	1500
5	MaxDrive	55	Gold	2400

Важно помнить:

Ключевое слово `AS` не меняет структуру самой таблицы в базе данных. Изменения происходят только в выводимом результате конкретного запроса.

16.2 Псевдонимы для таблиц (Сокращение кода)

Если имя таблицы длинное или в будущем мы начнем связывать несколько таблиц между собой, писать каждый раз полное имя перед столбцом становится неудобно. Мы можем присвоить короткую букву-псевдоним всей таблице в блоке `FROM`.

Давай присвоим таблице `players` короткий псевдоним `p`:

Наш запрос:

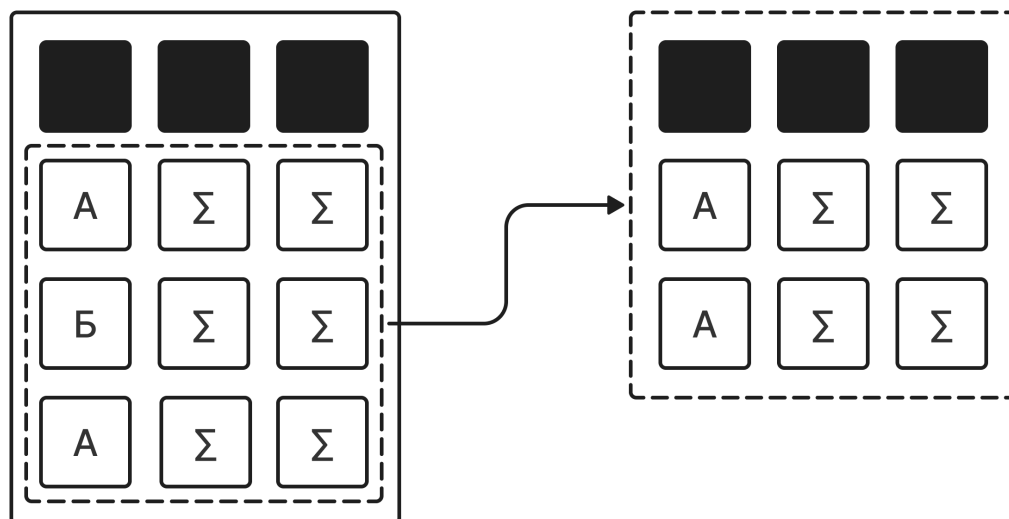
```
SELECT p.id,
       p.nickname AS player_name,
       p.city
FROM players AS p;
```

Как это работает:

Запись `players AS p` указывает SQL, что внутри этого запроса к таблице можно обращаться просто по букве `p`. Точка после псевдонима (`p.id`, `p.nickname`) явно указывает базе данных: «Возьми поле `id` именно из таблицы `p` (`players`)». Подобный подход считается стандартом хорошего тона у разработчиков, так как он защищает код от путаницы.

17 Группировка данных (Оператор GROUP BY)

Группировка в SQL позволяет объединять строки таблицы по определенному полю или списку полей, чтобы вычислить для них общие (агрегированные) значения: сумму, среднее, максимум, минимум или количество.



Чтобы сгруппировать данные по определенному столбцу, используется ключевое слово **GROUP BY**. Оно практически всегда работает в связке с **агрегатными функциями**:

- **COUNT** — подсчитывает количество строк.
- **AVG** — вычисляет среднее арифметическое.
- **SUM** — считает сумму значений.
- **MAX** — находит максимальное значение.
- **MIN** — находит минимальное значение.

Давай разберем, как анализировать показатели пользователей из нашей таблицы `players` с помощью группировки.

17.1 Среднее значение и округление (AVG, ROUND)

Представим, что разработчику нужно узнать, в каких городах живут самые опытные пользователи. Сгруппируем игроков по городам (`city`) и рассчитаем средний уровень (`level`) персонажей.

Наш запрос:

```
SELECT city,  
       AVG(level) AS avg_level  
FROM players  
GROUP BY city;
```

Результат (срез, первые 5 строк):

Поскольку среднее значение часто получается дробным с длинным хвостом, результат может выглядеть неаккуратно:

city	avg_level
Москва	43.5000
Санкт-Петербург	51.5000
Новосибирск	89.0000
Екатеринбург	54.0000
Омск	55.0000

Чтобы привести отчет в красивый вид, используем функцию `ROUND`. Она принимает два аргумента: что округлять и до скольки знаков после запятой.

Доработаем запрос:

```
SELECT city,  
       ROUND(AVG(level), 1) AS avg_level  
FROM players  
GROUP BY city;
```

Результат (срез, первые 5 строк):

city	avg_level
Москва	43.5
Санкт-Петербург	51.5
Новосибирск	89.0
Екатеринбург	54.0
Омск	55.0

17.2 Подсчет количества строк (COUNT)

Чтобы узнать, насколько равномерно распределена наша аудитория по игровым рангам, посчитаем количество игроков в разрезе каждого звания (`rank_title`).

Наш запрос:

```
SELECT rank_title,  
       COUNT(id) AS players_count  
FROM players  
GROUP BY rank_title;
```

Результат:

rank_title	players_count
Gold	16
Bronze	8
Diamond	3
Silver	13
Platinum	9
Master	1

17.3 Фильтрация групп (Оператор HAVING)

Что делать, если нам интересны только многочисленные ранги (например, где состоит больше 10 игроков)? Обычный оператор `WHERE` здесь не поможет, потому что он работает с отдельными строками еще до того, как они объединились в группы.

Для фильтрации уже сгруппированных данных используется оператор `HAVING`.

Наш запрос:

```
SELECT rank_title,  
       COUNT(id) AS players_count  
FROM players  
GROUP BY rank_title  
HAVING COUNT(id) > 10;
```

Результат:

Группы с рангом `Diamond` автоматически отсеялись:

rank_title	players_count
Gold	16
Silver	13

17.4 Сортировка сгруппированных данных

Итоговый отчет можно дополнительно отсортировать по алфавиту или по результату вычислений с помощью уже знакомого оператора `ORDER BY`, который пишется в самом конце запроса.

Наш запрос:

```
SELECT rank_title,  
       COUNT(id) AS players_count  
FROM players  
GROUP BY rank_title  
HAVING COUNT(id) > 10  
ORDER BY players_count DESC;
```

Результат:

rank_title	players_count
Gold	16
Silver	13

17.5 Поиск максимума и минимума (MAX, MIN)

Давай посмотрим на распределение соревновательного рейтинга (`rating`) внутри каждой гильдии (`guild`). Выведем одновременно максимальный и минимальный показатель рейтинга для каждого клана:

Наш запрос:

```
SELECT guild,  
       MAX(rating) AS max_rating,  
       MIN(rating) AS min_rating  
FROM players  
WHERE guild IS NOT NULL  
GROUP BY guild;
```

Результат:

guild	max_rating	min_rating
Грифоны Эрафии	2800	1750
Черные Драконы Нигона	3600	2350
Золотые Единороги Авли	2200	1200
Церберы Инферно	2950	1600
Гидры Таталии	2150	1450
Громовые Птицы Крюлода	2600	2500
Фениксы Конфлюкса	3100	1250
Древние Бегемоты	950	950
Мантикоры Подземелья	2850	2750

17.6 Группировка по нескольким полям

В SQL можно разделять данные на более мелкие подгруппы, указав несколько полей через запятую.

Например, давай узнаем, сколько игроков определенного уровня зарегистрировалось в конкретные дни. Сгруппируем таблицу по дате регистрации (`registration_date`) и уровню (`level`):

Наш запрос:

```
SELECT registration_date,  
       level,  
       COUNT(id) AS players_count  
FROM players  
GROUP BY registration_date,  
       level;
```

Результат (срез, первые 5 строк):

registration_date	level	players_count
2025-01-15	45	1
2026-05-10	12	1
2024-08-22	89	1
2025-11-03	31	1
2025-03-14	55	1

17.7 Разница между WHERE и HAVING: золотое правило

Главное отличие, которое нужно запомнить раз и навсегда:

WHERE

Выполняет фильтрацию **ДО** группировки.

Он отбирает отдельные строки таблицы.

HAVING

Выполняет фильтрацию **ПОСЛЕ** группировки.

Он работает с уже сформированными группами и чаще всего используется с агрегатными функциями.

18 Агрегатные функции

До этого момента мы учились выбирать отдельные строки из базы данных, фильтровать их и сортировать. Но в реальной работе аналитика или разработчика гораздо чаще возникает другая задача — получить **сводную статистику**.

Например, руководству игры неинтересно смотреть на список из 50 000 игроков по отдельности. Им нужно знать ответы на глобальные вопросы: «Сколько всего пользователей зарегистрировано? Какой у них средний уровень? Какое максимальное количество побед набрал лучший игрок?»

Для ответов на такие вопросы в SQL используются **агрегатные функции**.

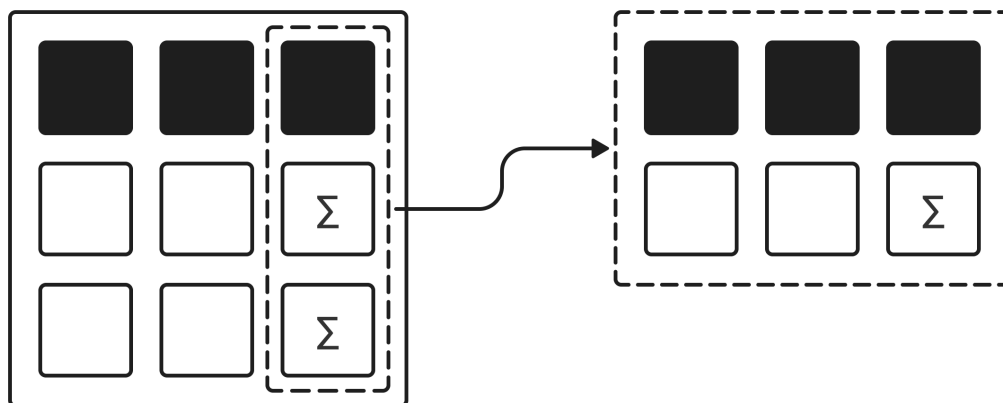
18.1 Что такое агрегатные функции и как они работают?

Представьте себе обычный чек из супермаркета.

Каждая строчка в нём — это отдельный товар и его цена. Это аналог обычного SQL-запроса, который выводит строки друг за другом.

А в самом низу чека всегда есть строчка, например: «**ИТОГО: 1500 рублей**». Чтобы получить это число, кассовый аппарат взял *всю пачку строк*, сложил цены и выдал *одно единственное значение*.

Именно это и делают агрегатные функции в SQL: они берут множество значений из колонки, обрабатывают их по определённому правилу и «схлопывают» в одну-единственную строку.



Мы разберем 5 самых главных функций, которые пригодятся вам в 95% задач:

1. **COUNT** — подсчёт количества строк.
2. **SUM** — подсчёт суммы чисел.
3. **AVG** — вычисление среднего значения.
4. **MIN** — поиск самого маленького значения.
5. **MAX** — поиск самого большого значения.

Для работы мы используем нашу привычную таблицу игроков `players`.

18.2 Функция COUNT (Количество)

Считает, сколько всего строк или записей попало в ваш запрос. Например, используется в виде `COUNT(*)`, что означает «посчитай вообще все строки».

Пример: Мы хотим узнать, сколько всего игроков зарегистрировано в нашей игровой базе данных.

```
SELECT COUNT(*)  
FROM players;
```

Результат выполнения:

COUNT(*)
50

(Функция вернула число 50, так как в нашей таблице сейчас ровно 50 игроков).

18.3 Функция SUM (Сумма)

Складывает все числа в выбранной колонке. Работает только с числовыми данными (например: уровень, рейтинг, победы).

Пример: Нам нужно узнать, сколько суммарно побед одержали абсолютно все участники гильдии „Фениксы Конфлюкса“. Для этого мы объединим функцию `SUM` и уже знакомую вам фильтрацию `WHERE`:

```
SELECT SUM(wins)  
FROM players  
WHERE guild = 'Фениксы Конфлюкса';
```

Результат выполнения:

SUM(wins)
782

18.4 Функция AVG (Среднее значение)

Считает среднее арифметическое всех чисел в колонке (складывает их и делит на их количество).

Пример: Давайте выясним, какой средний рейтинг у игроков, которые уже успели заработать почетное звание „Gold“.

```
SELECT AVG(rating)
FROM players
WHERE rank_title = 'Gold';
```

Результат выполнения:

AVG(rating)
2161.8750

18.5 Функция MIN (Минимум)

Находит самое маленькое (минимальное) значение в колонке.

Пример: Мы хотим узнать, какой самый низкий игровой уровень среди всех зарегистрированных пользователей.

```
SELECT MIN(level)
FROM players;
```

Результат выполнения:

MIN(level)
5

18.6 Функция MAX (Максимум)

Работает противоположно предыдущей функции — находит самое большое (максимальное) значение в колонке.

Пример: Давайте найдем рекорд нашей игры — максимальное количество побед (`wins`), которое набрал один игрок.

```
SELECT MAX(wins)
FROM players;
```

Результат выполнения:

MAX(wins)
620

Агрегатные функции вычисляют итоговое значение по набору строк и возвращают один результат на каждую группу (или один результат для всей выборки без `GROUP BY`).

`WHERE` позволяет заранее ограничить набор строк, которые попадут в вычисления, тем самым уменьшая объём данных для агрегации.

19 Что дальше?

Вы сделали первый шаг в изучении SQL и познакомились с основами работы с реляционными базами данных. Теперь вы умеете получать данные из таблиц, фильтровать и сортировать результаты, использовать встроенные функции и писать первые SQL-запросы.

Однако это лишь начало пути.

Чтобы уверенно работать с базами данных и применять SQL в реальных проектах, важно продолжать обучение и, самое главное, регулярно практиковаться.

Для этого была создана последовательная серия курсов, которая позволяет шаг за шагом освоить язык SQL — от базовых запросов до продвинутых методов анализа данных.

19.1 SQL Введение

Если эта книга основана на одноимённом курсе, вы можете пройти его в интерактивном формате. Курс содержит практические задания, автоматическую проверку решений и дополнительные упражнения, которые помогут закрепить материал.

[Записаться на курс «SQL Введение»](#)

19.2 SQL для начинающих

Следующий курс посвящён более глубокому изучению языка SQL. Вы познакомитесь с новыми конструкциями, научитесь строить более сложные запросы и закрепите знания на большом количестве практических задач, используя единую учебную базу данных.

[Записаться на курс «SQL для начинающих»](#)

19.3 SQL Оконные функции

После освоения базового SQL вы сможете перейти к одной из самых востребованных тем современного анализа данных — оконным функциям. Курс познакомит вас с ранжированием, накопительными вычислениями, функциями смещения и другими мощными инструментами обработки данных.

[Записаться на курс «SQL Оконные функции»](#)

19.4 Практический тренажёр по SQL

Этот курс практически полностью состоит из задач различной сложности. Вы будете работать с несколькими базами данных, решать реальные практические задания и закреплять навыки, максимально приближённые к тем, которые используются в профессиональной деятельности.

[Записаться на курс «Практический тренажёр по SQL»](#)

19.5 Проверка знаний и подготовка к собеседованию по SQL

Курс предназначен для проверки знаний SQL и подготовки к собеседованиям на позиции, где требуется знание SQL. В рамках курса вы сможете оценить свои навыки по различным

темам. Каждый вопрос сопровождается несколькими вариантами ответа, из которых необходимо выбрать правильный.

[Записаться на курс «Проверка знаний и подготовка к собеседованию по SQL»](#)

Какой бы путь вы ни выбрали дальше, помните: SQL — это язык, который невозможно освоить только чтением. Чем больше запросов вы напишете самостоятельно, тем увереннее будете чувствовать себя при работе с реальными данными.

Желаю вам успехов в дальнейшем обучении и новых профессиональных достижений!

20 Об авторе

Меня зовут Сергей Романенко.

Я разработчик, автор книг и образовательных курсов.

Моя цель — объяснять сложные темы простым, последовательным и практическим языком. В своих книгах и курсах я делаю упор не только на теорию, но и на понимание того, как технологии применяются в реальной разработке.

Если эта книга оказалась для вас полезной, буду рад, если вы расскажете о ней коллегам. Это помогает создавать новые материалы и делать их еще лучше.

Полезные ссылки:

- GitHub: <https://github.com/Awilum>
- Telegram: <https://awilum.t.me>